

Comprehensive Database Study Material

1. Database Fundamentals

Introduction

A database is an organized collection of data, typically stored and accessed electronically from a computer system. Databases are designed to manage large amounts of information efficiently, allowing for data storage, retrieval, updating, and deletion with ease. They are foundational to modern applications, from websites to enterprise systems.

Practical Applications

- **Business Applications:** Customer relationship management (CRM), inventory tracking, and financial systems.
- **Web Applications:** Storing user data, content management systems (e.g., WordPress), and e-commerce platforms.
- **Analytics:** Data warehousing for business intelligence and reporting.
- **IoT:** Managing sensor data in real-time for smart devices.
- **Healthcare:** Storing patient records and medical histories.

Database vs Spreadsheet

- **Scalability:** Databases handle large datasets and concurrent users, unlike spreadsheets, which are limited in size and performance.
- **Data Integrity:** Databases enforce constraints (e.g., foreign keys), while spreadsheets rely on manual checks.
- **Querying:** Databases support complex queries (SQL/NoSQL), whereas spreadsheets use basic filtering.
- **Concurrency:** Databases manage multiple users editing data simultaneously; spreadsheets risk data conflicts.

Database Terminology

- **Data:** Raw facts or values (e.g., a number or text).
- **Record:** A single row in a table, representing one instance of data.
- **Field:** A single column in a table, representing one attribute.
- **Table:** A collection of records with a specific structure.
- **Schema:** The blueprint of a database, defining tables, fields, and relationships.
- **Attribute:** A property or characteristic of an entity (synonymous with field).
- **Tuple:** A single row or record in a relational database.

Types of Databases

- **Relational (SQL):** Uses tables with predefined schemas (e.g., MySQL, PostgreSQL). Ideal for structured data with clear relationships.
- **Non-Relational (NoSQL):**
 - **Document:** Stores data as JSON-like documents (e.g., MongoDB). Flexible for semi-structured data.
 - **Key-Value:** Simple key-value pairs for fast lookups (e.g., Redis, DynamoDB).
 - **Graph:** Represents data as nodes and edges, ideal for relationships (e.g., Neo4j).
 - **Column-family:** Optimized for large-scale analytics (e.g., Cassandra).
- **In-memory Databases:** Store data in RAM for speed (e.g., Redis). Used in caching and real-time applications.
- **Time-series Databases:** Optimized for time-stamped data (e.g., InfluxDB). Common in IoT and monitoring.
- **NewSQL Databases:** Combine SQL's structure with NoSQL's scalability (e.g., CockroachDB). Used for distributed systems.

2. Relational Databases & SQL

Introduction

◆ What can I help you build?



Relational databases organize data into tables with rows and columns, using SQL (Structured Query Language) for querying and management. They are widely used due to their structured approach and ability to enforce data integrity.

Practical Applications

- **Enterprise Systems:** Managing employee records, financial transactions, or supply chains.
- **Web Development:** Storing user profiles, posts, or product catalogs.
- **Analytics:** Generating reports and dashboards from structured data.

Basic SQL

DDL (Data Definition Language)

Used to define or modify database structures:

- **CREATE:** Defines new tables or databases (e.g., `CREATE TABLE users (id INT, name VARCHAR(50));`).
- **ALTER:** Modifies existing structures (e.g., `ALTER TABLE users ADD email VARCHAR(100);`).
- **DROP:** Deletes tables or databases (e.g., `DROP TABLE users;`).
- **TRUNCATE:** Removes all records from a table but keeps its structure (e.g., `TRUNCATE TABLE users;`).

DML (Data Manipulation Language)

Manages data within tables:

- **SELECT:** Retrieves data (e.g., `SELECT * FROM users;`).
- **INSERT:** Adds new records (e.g., `INSERT INTO users (id, name) VALUES (1, 'John');`).
- **UPDATE:** Modifies existing records (e.g., `UPDATE users SET name = 'Jane' WHERE id = 1;`).
- **DELETE:** Removes records (e.g., `DELETE FROM users WHERE id = 1;`).

DCL (Data Control Language)

Controls access and permissions:

- **GRANT:** Assigns permissions (e.g., `GRANT SELECT ON users TO 'user1';`).
- **REVOKE:** Removes permissions (e.g., `REVOKE SELECT ON users FROM 'user1';`).

TCL (Transaction Control Language)

Manages transactions:

- **COMMIT:** Saves changes (e.g., `COMMIT;`).
- **ROLLBACK:** Reverts changes (e.g., `ROLLBACK;`).
- **SAVEPOINT:** Sets a point to roll back to (e.g., `SAVEPOINT save1;`).

SQL Querying

- **Filtering with WHERE:** Narrows results (e.g., `SELECT * FROM users WHERE age > 18;`).
- **Sorting with ORDER BY:** Sorts results (e.g., `SELECT * FROM users ORDER BY name ASC;`).
- **Limiting with LIMIT, OFFSET:** Restricts row count (e.g., `SELECT * FROM users LIMIT 10 OFFSET 20;`).
- **Aliases using AS:** Renames columns or tables (e.g., `SELECT name AS username FROM users;`).
- **Aggregate Functions:**
 - `COUNT()`: Counts rows (e.g., `SELECT COUNT(*) FROM users;`).
 - `SUM()`, `AVG()`, `MAX()`, `MIN()`: Performs calculations (e.g., `SELECT AVG(salary) FROM employees;`).
- **GROUP BY and HAVING:** Groups rows and filters groups (e.g., `SELECT department, COUNT(*) FROM employees GROUP BY department HAVING COUNT(*) > 5;`).
- **Wildcards and LIKE:** Pattern matching (e.g., `SELECT * FROM users WHERE name LIKE 'J%';`).
- **Subqueries:** Nested queries (e.g., `SELECT * FROM users WHERE id IN (SELECT id FROM orders);`).
- **Case Statements:** Conditional logic (e.g., `SELECT name, CASE WHEN age < 18 THEN 'Minor' ELSE 'Adult' END AS status FROM users;`).

Joins

Combine data from multiple tables:

- **INNER JOIN:** Matches rows (e.g., `SELECT * FROM users INNER JOIN orders ON users.id = orders.user_id;`).
 - **LEFT JOIN:** Includes all rows from the left table (e.g., `SELECT * FROM users LEFT JOIN orders ON users.id = orders.user_id;`).
 - **RIGHT JOIN:** Includes all rows from the right table.
 - **FULL OUTER JOIN:** Includes all rows from both tables.
 - **Self Joins:** Joins a table with itself (e.g., `SELECT a.name, b.name FROM users a JOIN users b ON a.manager_id = b.id;`).
 - **Cross Joins:** Cartesian product of rows (e.g., `SELECT * FROM users CROSS JOIN orders;`).
-

3. Database Design

Introduction

Database design involves structuring data to ensure efficiency, integrity, and scalability. It uses tools like ERDs to model relationships and applies normalization to reduce redundancy.

Practical Applications

- **Efficient Data Storage:** Reduces storage costs and improves query performance.
- **Data Integrity:** Ensures accurate and consistent data.
- **Scalability:** Supports growing datasets and user loads.

ERD (Entity-Relationship Diagrams)

- **Entities:** Objects like "Customer" or "Order."
- **Attributes:** Properties like "CustomerID" or "OrderDate."
- **Relationships:** Connections like "Customer places Order."

Keys

- **Primary Key:** Uniquely identifies records (e.g., `CustomerID`).
- **Foreign Key:** Links tables (e.g., `Order.CustomerID` references `Customer.CustomerID`).
- **Candidate Key:** Potential unique identifiers.
- **Composite Key:** Multiple columns forming a unique key.

Referential Integrity

Ensures foreign keys reference valid primary keys, maintaining data consistency.

Relationships

- **One-to-One:** One record links to one record (e.g., Person to Passport).
- **One-to-Many:** One record links to multiple records (e.g., Customer to Orders).
- **Many-to-Many:** Requires a junction table (e.g., Students to Courses via Enrollment).

Normalization

Reduces redundancy:

- **1NF:** No repeating groups; all attributes are atomic.
- **2NF:** 1NF + no partial dependencies.
- **3NF:** 2NF + no transitive dependencies.
- **BCNF:** Stricter 3NF, ensuring all determinants are candidate keys.

Denormalization

Intentionally adds redundancy for performance (e.g., caching aggregated data).

Surrogate Keys vs Natural Keys

- **Surrogate:** System-generated keys (e.g., auto-incremented IDs).
- **Natural:** Keys from data (e.g., email addresses). Surrogates are preferred for stability.

Schema Design Best Practices

- Use meaningful names.
 - Enforce constraints (e.g., NOT NULL).
 - Optimize for common queries.
 - Plan for scalability.
-

4. Transactions & Concurrency

Introduction

Transactions ensure reliable data operations, while concurrency controls simultaneous access to prevent conflicts.

Practical Applications

- **Banking:** Ensures accurate fund transfers.
- **E-commerce:** Manages inventory updates during purchases.
- **Multi-user Systems:** Prevents data corruption in collaborative apps.

What is a Transaction?

A transaction is a sequence of operations treated as a single unit, either all succeeding or all failing.

ACID Properties

- **Atomicity:** All operations complete or none do.
- **Consistency:** Transactions maintain data integrity.
- **Isolation:** Transactions are independent until complete.
- **Durability:** Committed changes are permanent.

Concurrency Issues

- **Dirty Read:** Reading uncommitted data.
- **Non-repeatable Read:** Different data on repeated reads within a transaction.
- **Phantom Read:** New rows appear during a transaction.

Isolation Levels

- **Read Uncommitted:** Allows dirty reads.
- **Read Committed:** Prevents dirty reads.
- **Repeatable Read:** Prevents non-repeatable reads.
- **Serializable:** Highest isolation, prevents all issues but slowest.

Locks

- **Pessimistic:** Locks data during transactions (e.g., row-level locks).
- **Optimistic:** Assumes no conflicts, checks at commit.

Deadlocks

Occur when transactions wait for each other's locked resources. Resolved via timeouts or deadlock detection.

5. Indexing & Performance Tuning

Introduction

Indexes improve query performance by enabling faster data retrieval, while performance tuning optimizes database operations.

Practical Applications

- **Faster Queries:** Speeds up searches in large datasets.
- **Scalability:** Supports high-traffic applications.
- **User Experience:** Reduces latency in apps.

What is an Index?

An index is a data structure (e.g., B-Tree) that speeds up data retrieval.

Types of Indexes

- **B-Tree:** Default for most queries, supports range searches.
- **Hash:** Fast for equality searches, not for ranges.
- **Composite Index:** Covers multiple columns.
- **Unique Index:** Enforces uniqueness.
- **Full-Text Index:** Optimizes text searches (e.g., in search engines).

Pros and Cons of Indexes

- **Pros:** Faster queries, improved performance.
- **Cons:** Increased storage, slower writes.

Query Optimization

- **Execution Plan:** Shows how queries are executed (e.g., `EXPLAIN SELECT * FROM users;`).
 - **Caching:** Stores frequent query results.
 - **Partitioning:** Splits tables for faster access.
 - **Sharding:** Distributes data across servers.
 - **Materialized Views:** Precomputed query results.
 - **Denormalization:** Reduces joins for performance.
-

6. Stored Programs and Extensions

Introduction

Stored programs (procedures, functions, triggers) and extensions (views, cursors) enhance database functionality and automation.

Practical Applications

- **Automation:** Triggers for audit logs.
- **Reusability:** Stored procedures for complex logic.
- **Simplified Queries:** Views for recurring queries.

Stored Procedures

Reusable SQL scripts stored in the database (e.g., `CREATE PROCEDURE update_salary() BEGIN ... END;`).

Triggers

Automatic actions on events (e.g., `CREATE TRIGGER log_insert AFTER INSERT ON users ...;`).

Functions (UDFs)

Custom functions for calculations (e.g., `CREATE FUNCTION get_age(birthdate DATE) RETURNS INT ...;`).

Views

Virtual tables from queries (e.g., `CREATE VIEW active_users AS SELECT * FROM users WHERE active = 1;`).

Cursors

Iterate over query results in stored procedures.

Transactions in Stored Procedures

Ensure atomic operations (e.g., `BEGIN TRANSACTION; ... COMMIT;`).

7. Database Administration

Introduction

Database administration involves managing database performance, security, and availability.

Practical Applications

- **Security:** Protects sensitive data.
- **Reliability:** Ensures uptime and data recovery.
- **Performance:** Optimizes query execution.

User Roles & Permissions

Assign roles (e.g., admin, reader) and permissions (e.g., `GRANT SELECT ON table TO user;`).

Backup and Recovery

- **Full Backup:** Complete database copy.
- **Incremental:** Changes since last backup.
- **Differential:** Changes since last full backup.

Database Migration

Moving data between databases or versions, ensuring compatibility.

High Availability & Failover

Redundant systems to minimize downtime (e.g., standby servers).

Replication

- **Master-Slave:** One primary, multiple read-only replicas.
- **Multi-Master:** Multiple writable nodes.

Monitoring & Auditing

Track performance and security (e.g., slow query logs).

Logging & Slow Query Analysis

Identify and optimize slow queries using logs.

8. NoSQL Databases

Introduction

NoSQL databases handle unstructured or semi-structured data, offering flexibility and scalability.

Practical Applications

- **Big Data:** Managing large-scale, unstructured data.
- **Real-time Apps:** Social media, gaming, and IoT.
- **Analytics:** Fast querying of diverse data.

Types of NoSQL

- **Document:** JSON-like storage (e.g., MongoDB: `db.users.find()`).
- **Key-Value:** Simple lookups (e.g., Redis: `GET user:1`).
- **Columnar:** Analytics-focused (e.g., Cassandra).
- **Graph:** Relationship-focused (e.g., Neo4j: `MATCH (n:Person) RETURN n;`).

CAP Theorem

Balances Consistency, Availability, and Partition Tolerance (e.g., MongoDB prioritizes availability).

BASE Model

- **Basically Available:** Prioritizes uptime.
- **Soft State:** Data may be inconsistent temporarily.
- **Eventual Consistency:** Data syncs over time.

Schema-less Design

No fixed schema, allowing flexible data structures.

Querying NoSQL

Varies by database (e.g., MongoDB: `db.users.find({age: {$gt: 18}})`).

9. Distributed Databases & Scalability

Introduction

Distributed databases spread data across multiple nodes for scalability and fault tolerance.

Practical Applications

- **Global Apps:** Low-latency access worldwide.
- **High Traffic:** Handles millions of users.
- **Fault Tolerance:** Ensures uptime during failures.

Horizontal vs Vertical Scaling

- **Horizontal:** Add more servers (e.g., sharding).
- **Vertical:** Upgrade server hardware.

Sharding

Splits data across servers (e.g., by user ID range).

Replication and Consistency Models

- **Replication:** Copies data for redundancy.
- **Consistency Models:** Strong vs. eventual consistency.

Eventual Consistency

Data syncs over time, common in NoSQL.

Consistent Hashing

Distributes data evenly across nodes.

Consensus Algorithms

- **Paxos:** Ensures agreement in distributed systems.
- **Raft:** Simplified consensus for replication.

Cloud-based DBs

Managed services like AWS RDS, Google Firestore, or Azure CosmosDB.

10. Advanced Topics

Introduction

Advanced database topics address specialized use cases like analytics, security, and real-time processing.

Practical Applications

- **Analytics:** Data warehousing for insights.
- **Security:** Protects sensitive data.
- **Real-time:** Powers streaming and AI applications.

Data Warehousing

Stores historical data for analytics (e.g., Snowflake, Redshift).

OLAP vs OLTP

- **OLAP:** Analytical queries (e.g., reporting).
- **OLTP:** Transactional queries (e.g., banking).

Data Lakes

Store raw, unstructured data for analysis.

Time-Series Databases

Handle time-stamped data (e.g., InfluxDB for IoT).

Graph Databases

Model relationships (e.g., Neo4j: `MATCH (a:Person)-[:FRIEND]->(b) RETURN a, b;`).

Search Engines as DBs

Elasticsearch for full-text search.

Streaming Data

Kafka with databases for real-time processing.

Database Security & Encryption

Use SSL, role-based access, and encryption (e.g., AES).

GDPR/Data Compliance

Ensures data privacy and user consent.

AI/ML-Integrated Databases

Databases like MindsDB integrate machine learning.

Bonus Topics

JSON, XML in Databases

Store and query structured formats (e.g., PostgreSQL JSONB).

ORM (Object-Relational Mapping)

Maps objects to database tables (e.g., SQLAlchemy: `class User(Base): ...`).

Connection Pooling

Manages database connections for efficiency.

DB in Web Development

Integrates databases with back-end frameworks (e.g., Django with PostgreSQL).

ETL Pipelines

Extract, transform, and load data for analytics.

CDC (Change Data Capture)

Tracks and processes database changes in real-time.